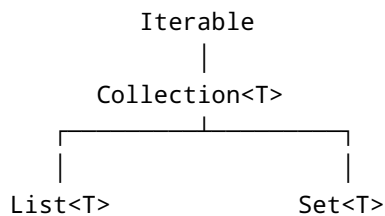


Kotlin Collections – Final Interview Notes

Collection Hierarchy



`Map<K, V>` is NOT part of the Collection hierarchy.

Why is `Map` not a `Collection`?

- `Collection` stores **individual elements**.
- `Map` stores **key-value pairs (entries)**.
- Therefore, `Map` has a separate hierarchy.

List

Definition

A `List` is an **ordered collection** that allows duplicate elements.

```
val list = listOf(10, 20, 20, 30)
```

Output

```
[10, 20, 20, 30]
```

Characteristics

- ☒ Ordered
- ☒ Allows duplicates
- ☒ Access by index
- ☒ Maintains insertion order

MutableList

```
val list = mutableListOf<Int>()

list.add(10)
list.add(20)
list.remove(10)
```

Default implementation:

ArrayList

Internal Implementation (ArrayList)

Internally:

```
ArrayList
    |
    ▼
+-----+-----+-----+-----+
| 10  | 20  | 30  | 40  |   |   |
+-----+-----+-----+-----+

size = 4

capacity = 5
```

When capacity becomes full:

1. Allocate a larger array (typically $\sim 1.5\times$ larger on the JVM).
2. Copy all existing elements.
3. Insert the new element.

Time Complexity

Operation	Complexity
Get(index)	$O(1)$

Operation	Complexity
Set(index)	O(1)
Add(end)	O(1) amortized
Add(middle)	O(n)
Remove	O(n)
Search	O(n)

Android Use Cases

- RecyclerView
 - Chat messages
 - Shopping cart
 - Image gallery
 - API response list
 - Search results
 - To-do list
-

Set

Definition

A `Set` stores **unique elements**.

```
val set = setOf(1,2,2,3)
```

Output

```
[1,2,3]
```

Duplicates are ignored.

Characteristics

-  No duplicate elements
 -  No index access
 -  Fast lookup
 -  Supports contains()
-

MutableSet

```
val set = mutableSetOf<String>()

set.add("Android")
set.remove("Android")
```

Default implementation

LinkedHashSet

LinkedHashSet Internal Structure

A LinkedHashSet combines:

Hash Table

+

Doubly Linked List

Hash Table

Bucket 2 → Node(10)

Bucket 5 → Node(20)

Bucket 7 → Node(30)

|

▼

Head

|

▼

10 ↔ 20 ↔ 30

▲

|

Tail

Why?

Hash Table

- Fast contains()
- Fast add()
- Fast remove()

Linked List

- Preserves insertion order

Time Complexity

Operation	Complexity
Add	$O(1)$ average
Remove	$O(1)$ average
Contains	$O(1)$ average
Iterate	$O(n)$

Android Use Cases

- Unique Bluetooth devices
- Selected item IDs
- Permission names
- Recently viewed screens
- Unique tags
- Unique notification IDs

Map

Definition

A Map stores

Key

↓

Value

Example

```
val map = mapOf(  
    1 to "Android",  
    2 to "Kotlin"  
)
```

Characteristics

- Keys are unique
- Values may repeat
- Fast lookup using key
- No index access

MutableMap

```
val map = mutableMapOf<Int,String>()  
  
map[1] = "Android"  
  
map.remove(1)
```

Default implementation

LinkedHashMap

LinkedHashMap Internal Structure

A LinkedHashMap combines

Hash Table

+

Doubly Linked List

Hash Table

Bucket 2 → Node

Bucket 5 → Node

Bucket 8 → Node

|



Head

|



1=A

↕

2=B

↕

3=C



|

Tail

Hash Table

- Fast get()
- Fast put()
- Fast remove()

Linked List

- Preserves insertion order

HashMap Internals

HashMap contains

Bucket Array

↓

Bucket

↓

Node

↓

Node

↓

Node

Each node stores

hash

key

value

next

Conceptually

```
class Node<K,V>{  
    int hash;  
  
    K key;  
  
    V value;  
  
    Node<K,V> next;  
}
```


Hash Code

Example

```
Key
↓
hashCode()
↓
Bucket Index
↓
Store Node
```

The hash determines which bucket the key belongs to.

Collision

Two keys may map to the same bucket.

```
Bucket 5
↓
(100, "Android")
↓
(200, "Kotlin")
↓
(300, "Compose")
```

This is called a collision.

Java 8 Optimization

If a bucket grows beyond a threshold (typically 8 entries, when the table is large enough):

Linked List

↓

Red-Black Tree

Search improves from

$O(n)$

↓

$O(\log n)$

Load Factor

Default

0.75

Example

Capacity = 16

Threshold =

16×0.75

=

12

When the threshold is exceeded, HashMap resizes.

Resizing

16

↓

32

↓

64

↓

128

Every existing entry is rehashed and redistributed into the new bucket array.

Resize Complexity

 $O(n)$

HashMap Complexity

Operation	Complexity
Put	$O(1)$ average
Get	$O(1)$ average
Remove	$O(1)$ average
ContainsKey	$O(1)$ average
Resize	$O(n)$
Worst-case lookup	$O(n)$ (or $O(\log n)$ for treeified buckets)

Comparison Table

Feature	List	Set	Map
Stores	Values	Unique Values	Key → Value
Duplicate Values	✓ Yes	✗ No	✓ Values only
Duplicate Keys	N/A	N/A	✗ No
Ordered	✓ Yes	LinkedHashSet : Yes	LinkedHashMap : Yes
Access by Index	✓ Yes	✗ No	✗ No
Access by Key	✗ No	✗ No	✓ Yes
Search Speed	$O(n)$	$O(1)$ average	$O(1)$ average

Feature	List	Set	Map
Default Mutable Implementation	ArrayList	LinkedHashSet	LinkedHashMap
Internal Structure	Dynamic Array	Hash Table + Doubly Linked List	Hash Table + Doubly Linked List

Which Collection Should You Choose?

Requirement	Recommended Collection
Ordered data with duplicates	List
Unique values	Set
Fast lookup by key	Map
RecyclerView items	List
Unique BLE devices	Set
API response cache	Map
Shopping cart	List
User ID → User object	Map
Selected IDs	Set
Chat messages	List

Interview Summary

List

- Ordered collection
- Allows duplicates
- Backed by `ArrayList`
- Fast random access using indexes
- Best for sequential UI data such as RecyclerView

Set

- Stores only unique elements
- Backed by `LinkedHashSet`
- Internally uses a **Hash Table + Doubly Linked List**
- Best when uniqueness is required

Map

- Stores key-value pairs
- Backed by `LinkedHashMap`
- Internally uses a **Hash Table + Doubly Linked List**
- Provides fast lookup by key

HashMap

- Uses **hash codes** to determine bucket placement
- Handles collisions using linked lists (or Red-Black Trees for large buckets in Java 8+)
- Resizes when the load factor threshold is exceeded
- `get()` and `put()` are **O(1)** on average because they compute the bucket directly instead of scanning all entries.

Easy Way to Remember

Collection	Think Of It As
List	 A numbered list where duplicates are allowed.
Set	 A unique guest list—each name appears only once.
Map	 A dictionary where each unique key points to a value.